

IST604 – Web Services & Distributed Computing

Chapter 2: Structural & Essay Practice Questions

These questions are designed for master's-level examination preparation. Structural questions test precise, concise knowledge; essay questions require critical analysis, comparison, and synthesis.

Part I: Structural / Short-Answer Questions

Group 1: Definitions and Concepts

Q1. Define a web service and list **four** key characteristics that distinguish it from other software components. (4 marks)

Model Answer Guidance: Define as a standards-based, language-agnostic software entity operating over a network. Key characteristics: standards-based, vendor-neutral, language-agnostic, transport-neutral, formatted requests, application-specific responses, remote operation.

Q2. Differentiate between a **web service** and an **API** with respect to: (a) network dependency, and (b) scope of applicability. (4 marks)

Model Answer Guidance: (a) Web services always require a network; APIs can be local. (b) All web services are APIs; not all APIs are web services.

Q3. Identify and briefly explain the **three core functionality features** of web services. (6 marks)

Model Answer Guidance: (1) Interoperability — bridges different languages/platforms. (2) Standardized Messaging — XML/JSON formats. (3) Loose Coupling — client/server independence; internal changes don't affect the interface.

Q4. State **three benefits** of using web services in enterprise application integration. (3 marks)

Model Answer Guidance: Loose coupling, ease of integration (breaking data silos), service reuse (code once, use many times).

Group 2: SOA and Architecture

Q5. With the aid of a labeled diagram, explain the **three-role SOA model** for web services, describing the responsibility of each role. (9 marks)

Model Answer Guidance: Diagram should show Service Provider → Registry ← Service Requester with "publish," "find," and "bind/invoke" arrows. Provider: develops and publishes interfaces. Broker/Registry: stores and enables discovery (UDDI). Requester/Consumer: finds and invokes services.

Q6. Compare the **RPC-based** and **Messaging-based** communication models for web services across the following dimensions: coupling, synchronicity, and data transmission unit. (6 marks)

Model Answer Guidance:

- Coupling: RPC = tightly coupled; Messaging = loosely coupled
- Synchronicity: RPC = synchronous (client waits); Messaging = asynchronous (client does not wait)
- Data unit: RPC = parameters/method calls; Messaging = entire documents

Q7. What is the role of **UDDI** in a Service-Oriented Architecture? How does it interact with WSDL and SOAP? (6 marks)

Model Answer Guidance: UDDI is the registry where providers publish and consumers discover services. Querying UDDI returns a WSDL description of the service interface. Using that WSDL, a developer constructs a SOAP client to communicate with the provider — forming the full publish-find-bind workflow.

Group 3: Standards — SOAP, WSDL, REST

Q8. List and describe the **four structural elements** of a SOAP message, indicating which are optional and which are required. (8 marks)

Model Answer Guidance: (1) Envelope — required, root element, identifies the document. (2) Header — optional, metadata/processing instructions. (3) Body — required, actual request/response payload. (4) Fault — optional (inside Body), describes exceptions. Bonus: Attachments (MIME encoded binary data).

Q9. Describe the **six-step SOAP exchange mechanism** between a client and a server. (6 marks)

Model Answer Guidance: (1) Client creates SOAP request. (2) Request contains Envelope with Header/Body. (3) Server receives and processes (validation, DB, business logic). (4) Server creates SOAP response using same protocol. (5) Response contains Envelope and payload. (6) Client receives and processes response.

Q10. Describe the **seven key elements** of a WSDL document, explaining the purpose of each. (14 marks)

Model Answer Guidance: `<definitions>` (root, namespaces), `<types>` (XSD data types), `<message>` (data units/parameters), `<portType>/<interface>` (abstract operations grouping), `<binding>` (protocol + message format), `<service>` (groups endpoints), `<port>/<endpoint>` (physical URL address).

Q11. Distinguish between the **Top-Down (Contract-First)** and **Bottom-Up (Code-First)** approaches to WSDL development, including an example tool for each. (4 marks)

Model Answer Guidance: Top-Down: start with WSDL file → generate code scaffolding (tools: `wsimport`, `WSDL2Java`). Bottom-Up: start with Java/code → auto-generate WSDL (tools: `wsgen`,

Java2WSDL).

Q12. List the **four WSDL operational patterns** and briefly describe the message flow for each. (8 marks)

Model Answer Guidance: One-way (receive, no response), Request-Response (receive + respond — most common), Solicit-Response (service initiates, waits for reply), Notification (service sends, no expectation of reply).

Group 4: Java Web Services

Q13. Name and distinguish the **two main Java APIs** for web services development, specifying the web service style each targets and their primary implementations. (6 marks)

Model Answer Guidance: JAX-WS — SOAP services, supports RPC and Document styles, available from Java 6+. JAX-RS — RESTful services, main implementations: Jersey and RESTeasy. Spring Boot is the modern preferred framework but not a formal standard.

Part II: Essay Questions

Essay 1 — Comparative Analysis

Q14. *"The choice between SOAP and REST is not merely a technical decision but a strategic one, shaped by security requirements, performance constraints, and system context."*

Critically compare SOAP and REST as approaches to building web services, covering: data formats, transport protocols, state management, security models, performance characteristics, and error handling. Conclude with a justified recommendation for each of the following scenarios: (a) a mobile banking application, (b) a public social media API. (25 marks)

Essay Guidance:

- Introduction: Frame SOAP as a strict protocol vs. REST as a flexible architectural style.
- Data formats: SOAP (XML only, verbose envelopes) vs. REST (JSON/XML/HTML, lightweight).
- Transport: SOAP (HTTP, SMTP, JMS, TCP) vs. REST (HTTP/HTTPS only).
- State: SOAP (can be stateful or stateless) vs. REST (inherently stateless).
- Security: SOAP (WS-Security, message-level encryption, ACID compliance) vs. REST (HTTPS, OAuth 2.0, JWT — transport-level).
- Performance: REST faster (JSON, caching); SOAP slower (XML overhead).
- Error handling: SOAP uses `<Fault>` element (structured, standardized); REST uses HTTP status codes (404, 500) + custom bodies.
- Scenario (a): SOAP — banking requires ACID, end-to-end encryption, formal contracts.
- Scenario (b): REST — public API prioritizes speed, scalability, ease of consumption.

Essay 2 — Architecture and Design

Q15. Describe the architecture of web services as realized through Service-Oriented Architecture (SOA). In your answer, explain the SOA roles and their interactions, the role of SOAP, WSDL, and UDDI as enabling

technologies, and discuss the practical steps involved in implementing a web service from design to consumption. (25 marks)

Essay Guidance:

- Introduction: SOA as an architectural style for independent, reusable, business-oriented services.
- Three SOA roles: Provider (develops, deploys, publishes via WSDL to UDDI), Broker/Registry (UDDI: stores service descriptions, enables discovery), Requester (finds via UDDI, reads WSDL, builds SOAP client, invokes service).
- How SOAP, WSDL, UDDI interlock: WSDL describes the service → UDDI registers it → SOAP carries the messages.
- Implementation lifecycle: Requirements/design → coding (JAX-WS/JAX-RS, annotations) → deployment (WebLogic, GlassFish) → testing (SoapUI, Postman) → discovery/consumption (UDDI registration, client proxy generation).
- Critical reflection: limitations of UDDI in modern practice; REST/OpenAPI as the contemporary alternative to SOAP/WSDL/UDDI.

Essay 3 — Standards and Document Structure

Q16. Web services communication relies on a layered set of standards. Discuss the primary web services standards (SOAP, WSDL, UDDI, REST, XML, JSON), explaining how they interact in a complete web services ecosystem. Include in your discussion the structure of a SOAP message and the structure of a WSDL document. (25 marks)

Essay Guidance:

- XML as the foundational data format; JSON as its modern lightweight counterpart.
- SOAP: built on XML, transported via HTTP; structure (Envelope → Header + Body + Fault).
- WSDL: XML-based contract; elements (types, message, portType, binding, service, port); both WSDL 1.1 and 2.0.
- UDDI: XML-based registry; publish-find-bind cycle; how querying UDDI returns WSDL.
- REST: architectural style; uses HTTP methods + JSON/XML; stateless; OpenAPI as its contract equivalent.
- Ecosystem interaction diagram: Provider publishes WSDL to UDDI → Requester queries UDDI → gets WSDL → builds SOAP client → sends SOAP messages over HTTP.
- Comparison of SOAP/WSDL/UDDI stack vs. REST/JSON/OpenAPI stack.

Essay 4 — Critical Reflection

Q17. Web services are described as being "language-agnostic, vendor-neutral, and transport-neutral." To what extent do these properties hold true in practice? Discuss with reference to the architectural models, communication standards, and real-world implementation challenges covered in this chapter. (20 marks)

Essay Guidance:

- Language-agnostic in theory: Java, .NET, PHP can all communicate via SOAP/REST.
- Vendor-neutral: standards (W3C, UDDI) reduce lock-in, but application server choices (WebLogic vs. GlassFish) can introduce vendor-specific behavior.

- Transport-neutral: SOAP supports HTTP, SMTP, JMS, TCP; REST is HTTP-only — REST therefore reduces transport neutrality.
- In practice: differences in WSDL interpretation across frameworks; serialization/deserialization discrepancies; tight coupling in RPC-based services challenges loose coupling ideals.
- Conclusion: The ideals hold strongly at the protocol/standard level but face practical constraints at the implementation and deployment level.

End of Practice Questions